

7

Building a Location Tracking App Using GPS and Maps

In this chapter, we will create a location tracking app that saves the location of the user and displays it as a heat map. We will learn how to run tasks in the background on iOS, macOS, and Android devices. We will extend the .NET MAUI **Map** control to display the map with the saved locations directly in the map.

The following topics will be covered in this chapter:

- Tracking the location of a user in the background on an iOS device and a macOS device
- Tracking the location of a user in the background on an Android device
- How to show maps in a .NET MAUI app
- How to extend the functionality of .NET MAUI maps

Let's get started!

Technical requirements

To be able to complete this project, you'll need to have Visual Studio for Mac or Windows installed, as well as the .NET MAUI components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. To build an iOS app using Visual Studio for Windows, you must have a Mac connected. If you don't have access to a Mac at all, you can just complete the Android part of this project.

You can find the full source for the code in this chapter at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition>.

IMPORTANT INFORMATION FOR WINDOWS USERS

*At the time of writing, there was no **Map** control for the Windows platform in .NET MAUI. This is due to the lack of a **Map** control in the underlying WinUI platform. For the latest information on **Map** support in Windows, visit the **Map** documentation at <https://learn.microsoft.com/en-us/dotnet/maui/user-interface/controls/map>.*

Project overview

Many apps can be made richer by adding a map and location services. In this project, we will build a location tracking app that we will call **MeTracker**. This app will track the position of the user and save it to an SQLite database so that we can visualize the result in the form of a heat map. To build this app, we will learn how to set up processes in the background on iOS, macOS, and Android. Luckily for us, the iOS and macOS implementations are identical; however, the Android implementation is very different. For the map, we will use the .NET MAUI **Maps** component and extend its functionality to build a heat map.

Due to the lack of **Map** support on Windows, and just for some variety, this chapter will use Visual Studio for Mac screenshots and references. If you don't have a Mac, don't worry; you can still complete the project for Android on your Windows development machine. If you need help with the steps, look at some of the earlier chapters for equivalent steps.

The estimated build time for this project is 180 minutes.

Building the MeTracker app

It's time to start building the app. Use the following steps to create a project from a template:

1. Open Visual Studio for Mac and click **New**:

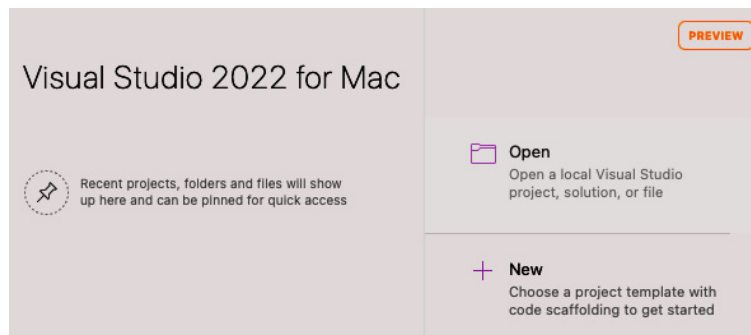


Figure 7.1 – Visual Studio for Mac start screen

2. In the **Choose a template for your new project** dialog, use the **.NET MAUI App** template, which is under **Multiplatform | App**; then, click **Continue**:

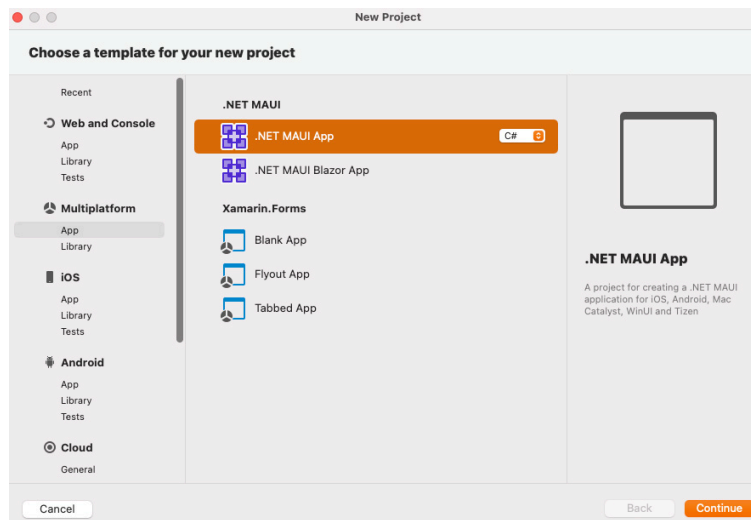


Figure 7.2 – New project

3. In the **Configure your new .NET MAUI App** dialog, ensure the **.NET 7.0** target framework is selected, then click **Continue**:



Figure 7.3 – Choosing a target framework

4. In the **Configure your new .NET MAUI App** dialog, name the project **MeTracker**, and then click **Create**:

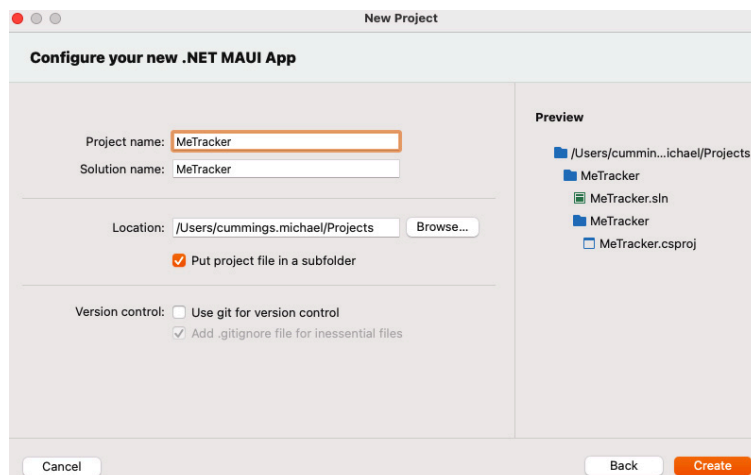


Figure 7.4 – Naming the new app

If you run the app now, you should see something like the following:

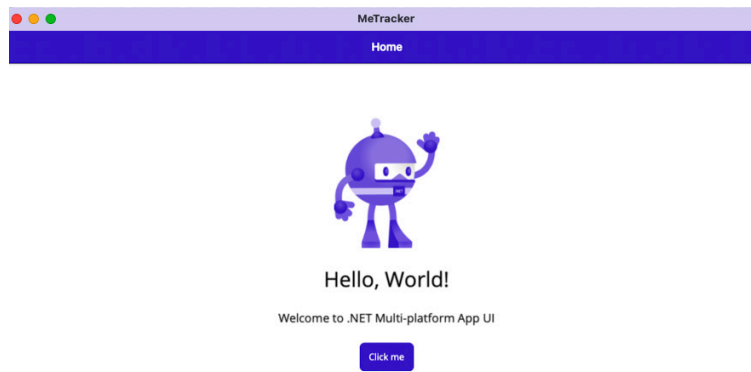


Figure 7.5 – MeTracker app on macOS

Now that we have created a project from a template, it's time to start coding!

Creating a repository to save the locations of the users

The first thing we will do is create a repository that we can use to save the locations of the users.

Creating a model for the location data

Before we create a repository, we will create a model class that will represent a user location. Follow these steps to do so:

1. Create a new **Models** folder that we can use for all our models.
2. Create a **Location** class in the **Models** folder and add properties for **Id**, **Latitude**, and **Longitude**.
3. Create two constructors – one that's empty and one that takes **latitude** and **longitude** as arguments. Use the following code to do so:

```
Using System;
namespace MeTracker.Models;
public class Location
{
    public Location() {}
    public Location(double latitude, double longitude)
    {
        Latitude = latitude;
        Longitude = longitude;
    }
    public int Id { get; set; }
    public double Latitude { get; set; }
    public double Longitude { get; set; }
}
```

Now that we have created a model, we can start creating a repository.

Creating a repository

First, we will create an interface for the repository. Follow these steps to do so:

1. Create a new folder called **Repositories**.
2. In our new folder, create an interface called **ILocationRepository**.
3. Write the following code in the new file that we created for the interface:

```
using MeTracker.Models;
using System;
using System.Threading.Tasks;
namespace MeTracker.Repositories;
public interface ILocationRepository
{
    Task SaveAsync(Models.Location location);
}
```

Now that we have an interface, we need to create an implementation of it. Follow these steps to do so:

1. Create a new **LocationRepository** class in the **Repositories** folder.
2. Implement the **ILocationRepository** interface and add the **async** keyword to the **SaveAsync** method using the following code:

```
using System;
using System.Threading.Tasks;
using MeTracker.Models;
namespace MeTracker.Repositories;
public class LocationRepository : ILocationRepository
{
    public async Task SaveAsync(Models.Location location)
    {
    }
}
```

A WORD ON THE ASYNC SUFFIX

*You will see in this and many other chapters in this book the use of **Async** as a suffix on methods. Appending a suffix of **Async** to all asynchronous methods is a .NET convention. How do we know whether a method is asynchronous in an interface where you can't see the **async** keyword? It will most likely return a **Task** or **ValueTask** object. There are some cases where an asynchronous method will return **void**; however, that is frowned upon, as Stephen Cleary explains in his article at <https://msdn.microsoft.com/en-us/magazine/jj991977.aspx>, so you won't see it used in this book.*

To store the data, we will use an SQLite database and the **object-relational mapper (ORM)** known as SQLite-net so that we can write code against a domain model instead of using SQL to perform operations against the database. This is an open source library that was created by Frank A. Krueger. Let's set this up by going through the following steps:

1. Add a reference to **sqlite-net-pcl** by right-clicking the **Dependencies** node in **Solution Explorer**:

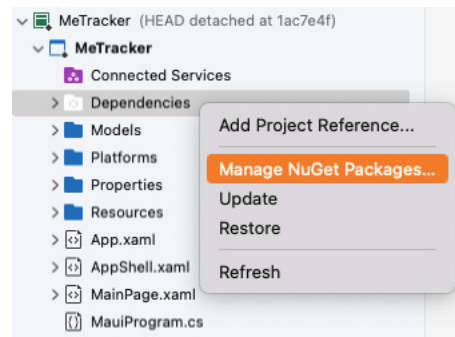


Figure 7.6 – Adding the NuGet package

1. Select **Manage NuGet Packages...** from the context menu to open the **NuGet Packages** window.
2. Check the **Include prereleases** checkbox and enter **sqlite-net-pcl** into the search box as shown next:

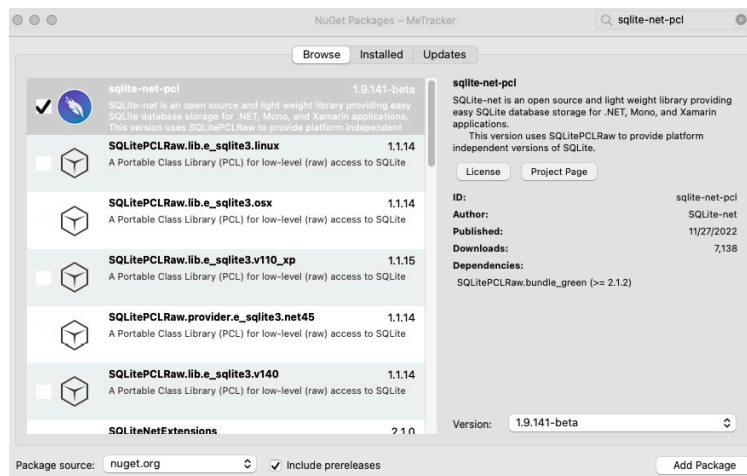


Figure 7.7 – Adding the sqlite-net-pcl package

3. Finally, check the box next to **sqlite-net-pcl** and click **Add Package**.
2. Go to the **Location** model class and add **PrimaryKeyAttribute** and **Auto IncrementAttribute** attributes to the **Id** property. When we add these attributes, the **Id** property will be a primary key in the database, and a value for it will be automatically created. The **Location** class should now look like the following:

```
using SQLite;
namespace MeTracker.Models;
public class Location
{
    public Location() { }
    public Location(double latitude, double longitude)
    {
        Latitude = latitude;
        Longitude = longitude;
    }
    [PrimaryKey]
    [AutoIncrement]
    public int Id { get; set; }
    public double Latitude { get; set; }
```

```
public double Longitude { get; set; }
}
```

3. Write the following code in the **LocationRepository** class to create a connection to the SQLite database. An **if** statement is used to check whether we have already created a connection. If we have, we won't create a new one; instead, we will use the connection that we've already created:

```
private SQLiteAsyncConnection connection;
private async Task CreateConnectionAsync()
{
    if (connection != null)
    {
        return;
    }
    var databasePath = Path.Combine(Environment.GetFolderPath (Environment.SpecialFolder .MyDoc
    connection = new SQLiteAsyncConnection(databasePath);
    await connection.CreateTableAsync<Location>();
}
```

Now, it's time to implement the **SaveAsync** method, which will take a **location** object as a parameter and store it in the database.

We will use the **CreateConnectionAsync** method in the **SaveAsync** method to ensure that a connection is created when we try to save data to the database. When we know that we have an active connection, we can just use the **InsertAsync** method and pass the **location** parameter of the **SaveAsync** method as an argument.

Edit the **SaveAsync** method in the **LocationRepository** class so that it looks like this:

```
public async Task SaveAsync(models.Location location)
{
    await CreateConnectionAsync();
    await connection.InsertAsync(location);
}
```

That wraps up the repository for now, so let's move on to the location tracking service.

Creating a service for location tracking

To track a user's location, we need to write code according to the platform. .NET MAUI has methods for getting the location of a user, but it cannot be used in the background. To be able to use the code that we will write for each platform, we need to create an interface. For the **ILocationRepository** interface, there is just one implementation that will be used on both platforms (iOS and Android), whereas for the location tracking service, we will have one implementation for each platform.

Go through the following steps to create an **ILocationTrackingService** interface:

1. Create a new folder called **Services**.
2. Create a new **ILocationTrackingService** interface in the **Services** folder.
3. In the interface, add a method called **StartTracking**, as shown in the following code snippet:

```
public interface ILocationTrackingService
{
    void StartTracking();
}
```

To make sure we can run and test our app while we implement the location tracking service for each platform, we will use a partial class. The main part of the class will be in the shared code section of the project and the platform-specific portions of the class will be in the platform-specific folders. We will come back to each implementation later in this chapter.

Create a class called **LocationTrackingService** in the **Services** folder, as shown:

```
public partial class LocationTrackingService : ILocationTrackingService
{
    public void StartTracking()
    {
        StartTrackingInternal();
    }
    partial void StartTrackingInternal();
}
```

We are using an interface to abstract our implementation. We are also using a partial class to abstract each specific implementation, but providing a base implementation so that we don't have to have an implementation for every platform immediately. However, the two methods (partial classes and base class inheritance) do not play together with the same method.

An implementation of the **StartTracking** interface method requires the **public** keyword, which would look like this:

```
public void StartTracking() {}
```

Then, make it partial, like so:

```
public partial void StartTracking() {}
```

The compiler complains that there is no initial definition of the partial method – that is, one that has no implementation.

Remove the empty definition, like so:

```
public partial void StartTracking();
```


The compiler now complains because it has an accessibility modifier, **public**.

There is just no making the compiler happy in this case. Therefore, to avoid these issues, we implement the **StartTracking** interface method by calling a **StartTrackingInternal** partial method. We will visit the implementation of **StartTrackingInternal** for each platform later in this chapter; for now, the app should compile and run, even though we haven't implemented **StartTrackingInternal**.

Now that we have the interface and base implementation of the location tracking service, we can turn our attention to the app logic and user interface.

Setting up the app logic

Now that we have created the interfaces, we need to track the location of the user and save it locally on the device. It's time to write some code so that we can start tracking a user. We still don't have any code that tracks the location of the user, but it will be easier to write this if we have already written the code that starts the tracking process.

Creating a view with a map

To start with, we will create a view with a simple map that is centered on the position of the user. Let's set this up by going through the following steps:

1. Create a new folder called **Views**.
2. In the **Views** folder, create a XAML-based **ContentPage** template and name it **MainView**:

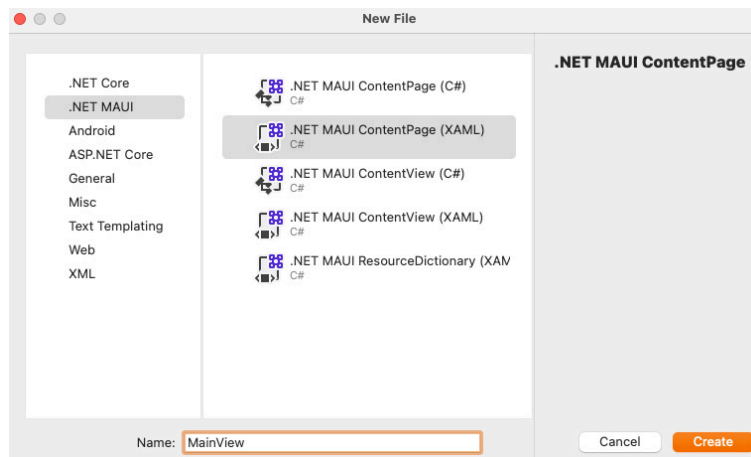


Figure 7.8 – Adding the .NET MAUI XAML ContentPage component

3. Add a reference to **Microsoft.Maui.Controls.Maps** by right-clicking the **Dependencies** node in **Solution Explorer**:

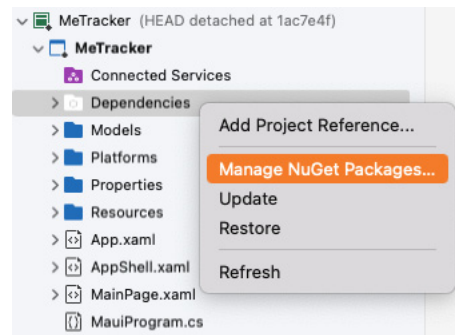


Figure 7.9 – Adding the NuGet package

1. Select **Manage NuGet Packages...** from the context menu to open the **NuGet package manager** window.
2. Type **Microsoft.Maui.Controls.Maps** into the search box as shown next:

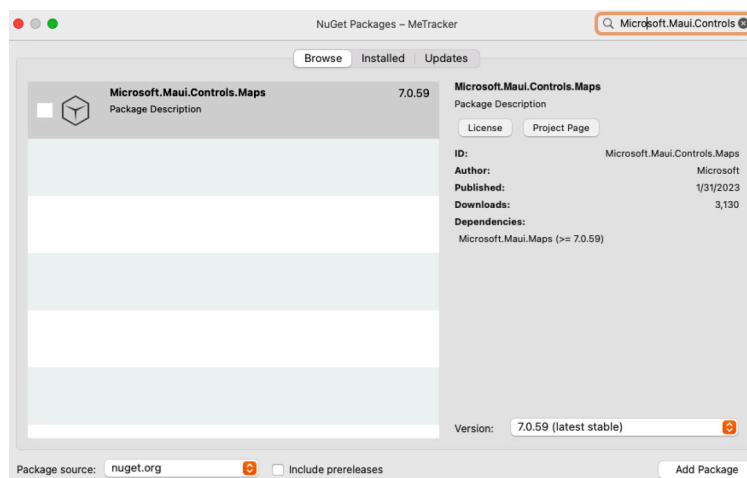


Figure 7.10 – Adding the .NET MAUI Maps package

3. Finally, check the box next to **Microsoft.Maui.Controls.Maps** and click **Add Package**.
4. Add the Map initialization code by opening the **MauiProgram.cs** file and making the highlighted change:

```
var builder = MauiApp.CreateBuilder();
builder
    .UseMauiApp<App>()
    .ConfigureFonts(fonts =>
    {
        fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
        fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
    })
    .UseMauiMaps();
return builder.Build();
```

5. Add the namespace for **Microsoft.Maui.Controls.Maps** to **MainView** using the following highlighted code:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:maps="clr-namespace:Microsoft.Maui.Controls.Maps;assembly=Microsoft.Maui.Controls.Map
```

```
x:Class="MeTracker.Views.MainView"
Title="MainView">
```

Now, we can use the map in our view. Because we want **Map** to cover the whole page, we can add it to the root of **ContentPage**.

6. Add **Map** to **ContentPage** with a name so that we can access it from the code-behind file. Name it **Map**, as shown in the following code snippet:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:maps="clr-namespace:Microsoft.Maui.Controls.Maps;assembly=Microsoft.Maui.Controls.Map"
             x:Class="MeTracker.Views.MainView"
             Title="MainView">
    <maps:Map x:Name="Map" />
</ContentPage>
```

Before we can start the app to see the **Map** control for the first time, we need to set the shell to use our new **MainView** template instead of the default **MainPage** template. But first, we will delete the **MainPage.xaml** and **MainPage.xaml.cs** files that we created when we started the project since we won't be using them here:

1. Delete the **MainPage.xaml** and **MainPage.xaml.cs** files in the project since we will be setting our **MainView** template as the first view that the user sees.
2. Edit the **AppShell.xaml** file, as shown in the following highlighted code:

```
<Shell
    x:Class="MeTracker.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:MeTracker"
    xmlns:views="clr-namespace:MeTracker.Views"
    Shell.FlyoutBehavior="Disabled">
    <ShellContent
        Title="Home"
        ContentTemplate="{DataTemplate views:MainView}"
        Route="MainView" />
</Shell>
```

Could we have used the existing **MainPage** template as it was? Sure – it really doesn't make any difference to the compiler what the XAML file is named or where it is located, but for consistency and by MVVM convention in .NET MAUI, we put our *pages* in the **Views** folder and suffix page names with **Views**.

Choosing either Mac Catalyst or an iOS simulator and running the app will produce the result shown in *Figure 7.11*. Android won't work until we have completed the next section:



Figure 7.11 – Running the app after adding the Map control

Now that we have a page with the **Map** control on it, we will need to make sure we have permission from the user to use location information.

Declaring platform-specific location permissions

To use the **Map** control, we need to declare that we require permission to location information. The **Map** control will make the runtime request if it is required. iOS/Mac Catalyst and Android each have their own way of declaring the required permissions. We will start with iOS/Mac Catalyst, following which we will do Android.

Open the **info.plist** file in the **Platforms/iOS** folder into **Property List Editor** by double-clicking on it. Add two new entries to the file, highlighted in the next screenshot:

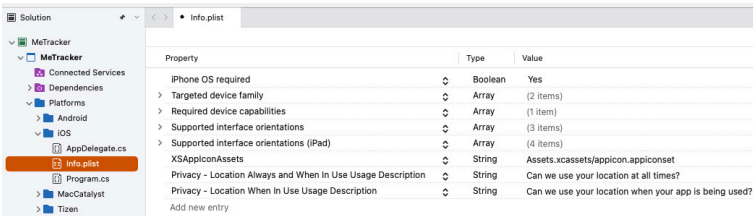


Figure 7.12 – Editing info.plist for iOS

Make the same changes in the **info.plist** file in the **Platforms/MacCatalyst** folder, as shown next:

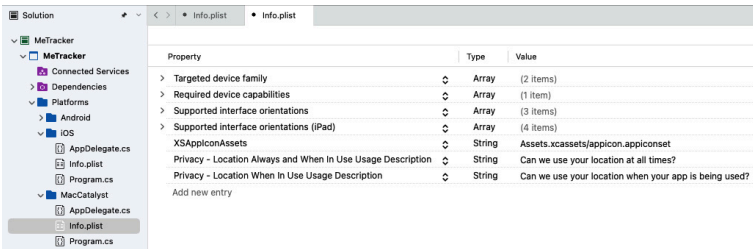


Figure 7.13 – Editing info.plist for Mac Catalyst

WINDOWS USERS

To edit the **info.plist** files on Windows, you need to open the file in a text editor by right-clicking the file, selecting **Open With...**, and then choosing **XML Editor**. Then, add the entries highlighted in the next code snippet.

Editing the **info.plist** file using the **Property List Editor** results in the changes highlighted next:

```
<key>XSAppIconAssets</key>
<string>Assets.xcassets/appicon.appiconset</string>
<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>Can we use your location at all times?</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Can we use your location when your app is being used?</string>
</dict>
</plist>
```

To track the location of the user in the background with Android, we need to declare five permissions, as shown in the following table:

ACCESS_COARSE_LOCATION	To get an approximate location for the user
ACCESS_FINE_LOCATION	To get a precise location for the user
ACCESS_NETWORK_STATE	We need this because the location services in Android use information from a network to determine the location of the user
ACCESS_WIFI_STATE	We need this because the location services in Android use information

from a Wi-Fi network to determine the location of the user

RECEIVE_BOOT_COMPLETED So that the background job can start again after the device is rebooted

The following steps will declare the required permissions for our app:

1. Open the **MainActivity.cs** file in the **Platforms/Android** folder.
2. After the **using** declarations' block method, add the following **assembly** attributes, highlighted next:

```
using Android.App;
using Android.Runtime;
[assembly: UsesPermission(Android.Manifest.Permission.AccessCoarseLocation)]
[assembly: UsesPermission(Android.Manifest.Permission.AccessFineLocation)]
[assembly: UsesPermission(Android.Manifest.Permission.AccessWifiState)]
[assembly: UsesPermission(Android.Manifest.Permission.ReceiveBootCompleted)]
namespace MeTracker;
```

Note that we don't declare

Android.Manifest.Permission.AccessNetworkState because it is part of the .NET MAUI template.

Now that we have declared all the permissions that we require, we can enable the map services on Android.

Android requires an **API key** for **Google Maps** to work with maps. The Microsoft documentation regarding how to obtain an API key can be found at <https://learn.microsoft.com/en-us/dotnet/maui/user-interface/controls/map?view=net-maui-7.0#get-a-google-maps-api-key>. Follow those instructions to obtain your Google Maps key, then use your key in the following steps to configure the Google Maps API key in the app:

1. Open **AndroidManifest.xml**, which is in the **Platforms/Android** folder, by right-clicking on the file and selecting **Open With...**, then selecting **XML (Text) Editor**.
2. Insert a metadata element as a child of the application element, as shown in the following highlighted code, replacing **"{YourKeyHere}"** with the key you obtained from Google:

```
<application android:label="MeTracker.Android">
  <meta-data android:name="com.google.android.geo.API_KEY" android:value="{YourKeyHere}" />
</application>
```

Recent versions of Android and iOS have changed how permissions are handled. Certain permissions such as location are not granted without explicit approval from the user while the app is running. It is also possible that the user can deny permissions. Let's look at how to handle runtime permission requests in the next section.

Requesting location permission at runtime

Before we can use the location of the user, we need to request permissions from the user. .NET MAUI has cross-platform permission APIs, and we just need a tiny bit of code to make handling the request a little nicer. To implement the permission request handling, follow these steps:

1. Create a new class called **AppPermissions** in the root of the project.
2. Edit the new file to look like the following:

```
namespace MeTracker;
internal partial class AppPermissions
{
    internal partial class AppPermission : Permissions.LocationWhenInUse
    {
    }
    public static async Task<PermissionStatus> CheckRequiredPermissionAsync() => await Permissions.CheckStatusAsync<AppPermission>();
    public static async Task<PermissionStatus> CheckAndRequestRequiredPermissionAsync()
    {
        PermissionStatus status = await Permissions.CheckStatusAsync<AppPermission>();
        if (status == PermissionStatus.Granted)
            return status;
        if (status == PermissionStatus.Denied && DeviceInfo.Platform == DevicePlatform.iOS)
        {
            // Prompt the user to turn on in settings
            // On iOS once a permission has been denied it may not be requested again from the
            await App.Current.MainPage.DisplayAlert("Required App Permissions", "Please enable
        }
        if (Permissions.ShouldShowRationale<AppPermission>())
        {
            // Prompt the user with additional information as to why the permission is needed
            await App.Current.MainPage.DisplayAlert("Required App Permissions", "This is a loca
        }
        status = await MainThread.InvokeOnMainThreadAsync(Permissions.RequestAsync<AppPermissio
        return status;
    }
}
```

This creates a type named **AppPermission** that derives from the default .NET MAUI **LocationWhenInUse** permission class.

The **CheckRequiredPermission** method is used to ensure our app has the right permissions before we attempt any operations that might fail if we don't. Its implementation is to call the .NET MAUI **CheckSyncStatus** method with our **AppPermission** type. It returns a **PermissionStatus** type, which is an enum. We are mostly interested in the **Denied** and **Granted** values.

The **CheckAndRequestRequiredPermission** method handles the intricacies of requesting access from the user. The first step is to simply check and see whether the permission has already been granted, and if it has, return the status. Next, if we are on iOS and the permission has been denied, it cannot be requested again, so you must instruct the user on how to grant permission to the app by using the settings panel. Android includes in the request behavior the ability to nag the user if they have denied access. This behavior is exposed through .NET MAUI with the **ShouldShowRationale** method. It will return **false** for any platform that does not support this behavior, and on Android, it will return **true** the first time the user denies access and **false** if the user denies it a second time. Finally, we request access to the

AppPermission type from the user. Again, .NET MAUI is hiding all the platform implementation details from us, making checking and requesting access to certain resources very straightforward.

Now that we have the **AppPermissions** class in place, we can use it to request the current location of the user, and center that map on that location.

Centering the map on the current user location

We will center the map on the position of the user in the constructor of **MainView.xaml.cs**. Because we want to fetch the user's location asynchronously and this needs to be executed on the main thread, we will use **MainThread.BeginInvokeOnMainThread** to run an anonymous method on the main thread. Once we have the location, we can use the **MoveToRegion** method of **Map**. We can set this up by going through the following steps:

1. Open **MainView.xaml.cs**.
2. Add the highlighted code shown in the following code snippet to the constructor of the **MainView.xaml.cs** class:

```
public MainView ()
{
    InitializeComponent();
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        var status = await AppPermissions.CheckAndRequestRequiredPermissionAsync();
        if (status == PermissionStatus.Granted)
        {
            var location = await Geolocation.GetLastKnownLocationAsync();
            if (location == null)
            {
                location = await Geolocation.GetLocationAsync();
            }
            Map.MoveToRegion(MapSpan.FromCenterAndRadius(
                location,
                Distance.FromKilometers(50)));
        }
    });
}
```

If you run the application now, it should look something like the following:



Figure 7.14 – Map centered on user location

Now that we have the map displaying our current location, let's start building the logic of the rest of the app, starting with our **ViewModel** class.

Creating a ViewModel class

Before we create an actual **ViewModel** class, we will create an abstract base view model that all view models can inherit from. The idea behind this base view model is that we can write common code in it. In this case, we will implement the **INotifyPropertyChanged** interface by using the **CommunityToolkit.Mvvm** NuGet package. To add the package, follow these steps:

1. Add a reference to **CommunityToolkit.Mvvm** by right-clicking the **Dependencies** node in **Solution Explorer**:

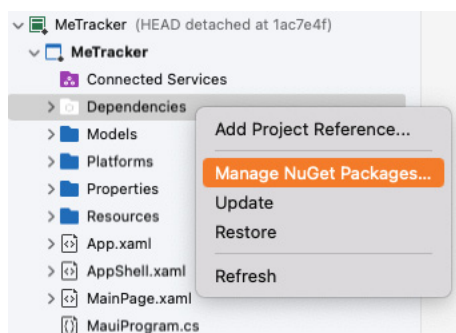


Figure 7.15 – Adding the NuGet package

2. Select **Manage NuGet Packages...** from the context menu to open the **NuGet package manager** window.
3. Type **CommunityToolkit.Mvvm** into the search box, as shown next:

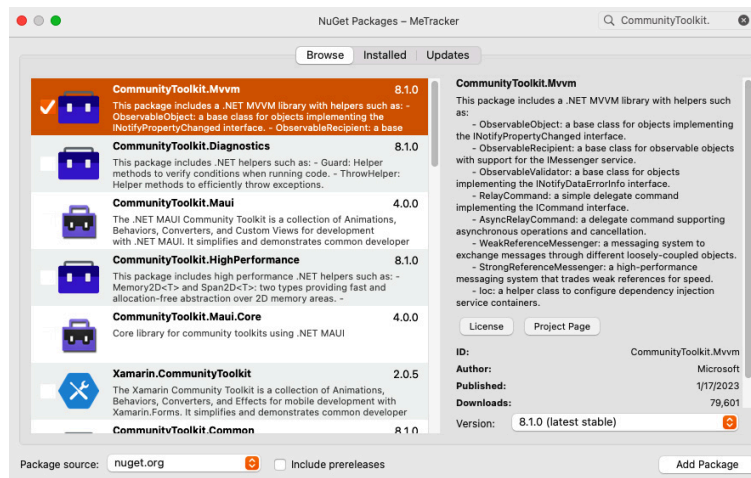


Figure 7.16 – Adding the CommunityToolkit.Mvvm package

4. Finally, check the box next to **CommunityToolkit.Mvvm** and click **Add Package**.

Now, we can create a **ViewModel** class by going through the following steps:

1. Create a folder called **ViewModels** in the project.
2. Create a new class called **ViewModel**.
3. Modify the template code to match the following:

```
using CommunityToolkit.Mvvm.ComponentModel;
namespace MeTracker.ViewModels;
public partial class ViewModel : ObservableObject
{
}
```

The next step is to create the actual view model that will use **ViewModel** as a base class. Let's set this up by going through the following steps:

1. Create a new **MainViewModel** class in the **ViewModels** folder.
2. Make the **MainViewModel** class inherit **ViewModel**.
3. Add a read-only field of the **ILocationTrackingService** type and name it **locationTrackingService**.
4. Add a read-only field of the **ILocationRepository** type and name it **locationRepository**.
5. Create a constructor with **ILocationTrackingService** and **ILocationRepository** as parameters.
6. Set the values of the fields that we created in *steps 3 and 4* with the values from the parameters, as shown in the following code snippet:

```
public class MainViewModel : ViewModel
{
    private readonly ILocationRepository locationRepository;
    private readonly ILocationTrackingService locationTrackingService;
    public MainViewModel(ILocationTrackingService locationTrackingService,
        ILocationRepository locationRepository)
    {
        this.locationTrackingService = locationTrackingService;
    }
}
```

```

        this.locationRepository = locationRepository;
    }
}

```

To make the app start tracking the location of a user, we need to run the code that starts the tracking process on the main thread. Follow these steps:

1. In the constructor of the newly created **MainViewModel** class, add an invocation to the main thread using **MainThread.BeginInvokeOnMainThread**.
2. Call **locationService.StartTracking** in the action that we pass to the **BeginInvokeOnMainThread** method. This is shown in the following highlighted code:

```

public MainViewModel(ILocationTrackingService locationTrackingService, ILocationRepository loca
{
    this.locationTrackingService = locationTrackingService;
    this.locationRepository = locationRepository;
    MainThread.BeginInvokeOnMainThread(() =>
    {
        locationTrackingService.StartTracking();
    });
}

```

Finally, we need to inject a **MainViewModel** class into the constructor of **MainView** and assign the **MainViewModel** instance to the binding context of the view. This will allow what data binding we've done to be processed, and the properties of **MainViewModel** will be bound to the controls in the user interface. Follow these steps:

1. Go to the constructor of the **Views/MainView.xaml.cs** file.
2. Add **MainViewModel** as a parameter of the constructor and call it **viewModel**.
3. Set **BindingContext** as the instance of **MainViewModel**, as shown in the following code snippet:

```

public MainView(MainViewModel viewModel)
{
    InitializeComponent();
    BindingContext = viewModel;
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        var location = await Geolocation.GetLastKnownLocationAsync();
        if(location == null)
        {
            location = await Geolocation.GetLocationAsync();
        }
        Map.MoveToRegion(MapSpan.FromCenterAndRadius(
            location, Distance.FromKilometers(5)));
    });
}

```

In order for .NET MAUI to locate the classes we have implemented in this section so far, we need to add them to the **dependency injection (DI)** container.

Adding classes to the DI container

Since we have added a parameter to the constructor of the view, the .NET MAUI **View** framework won't be able to construct the view automatically. So, we need to add **MainView**, **MainViewModel**, **LocationTrackingService**, and **LocationRepository** instances to the DI container. To do that, follow these steps:

1. Open the **MauiProgram.cs** file.
2. Add the following highlighted lines to the **CreateMauiApp** method:

```
public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();
    builder
        .UseMauiApp<App>()
        .ConfigureFonts(fonts =>
        {
            fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
        })
        .UseMauiMaps();
    #if DEBUG
        builder.Logging.AddDebug();
    #endif

    builder.Services.AddSingleton<Services.ILocationTrackingService, Services.LocationT
    builder.Services.AddSingleton<Repositories.ILocationRepository, Repositories.LocationRe
    builder.Services.AddTransient(typeof(ViewModels.MainViewModel));
    builder.Services.AddTransient(typeof(Views.MainView));
    return builder.Build();
}
```

Now, we will be able to run the app again. We haven't changed any of the interfaces, so it should look and behave the same as before. If it doesn't, go back through the previous section carefully to make sure you have all the code correct.

Let's add some code so that we can track the user's location over time using background location tracking.

Background location tracking on iOS and Mac Catalyst

The code for location tracking is something that we need to write for each platform. For iOS and Mac Catalyst, we will use **CLLocationManager** from the **CoreLocation** namespace.

Enabling location updates in the background

When we want to perform tasks in the background in an iOS or Mac Catalyst app, we need to declare what we want to do in the **info.plist**

file. The following steps show how we go about this:

1. Open **info.plist**; you will need to do this for both **Platforms/iOS/info.plist** and **Platforms/MacCatalyst/info.plist**.
2. Add the following highlighted entry using the **Property List Editor** by selecting **Required background modes** from the dropdown and **App registers for location updates**, as shown in the following screenshot:

XSAppIconAssets	⇅	String	Assets.xcassets/appicon.appiconset
Privacy - Location Always and When In U...	⇅	String	Can we use your location at all times?
Privacy - Location When In Use Usage D...	⇅	String	Can we use your location when your app is being used?
▼ Required background modes	⇅	Array	(1 item)
		String	App registers for location updates

Figure 7.17 – Adding location updates

We can also enable background modes directly in the **info.plist** file if we open it with an XML editor. In this case, we will add the following XML:

```
<key>UIBackgroundModes</key>
<array>
  <string>location</string>
</array>
```

Subscribing to location updates

Now that we have prepared the **info.plist** file for location tracking, it is time to write the actual code that will track the location of the user. If we don't set **CLLocationManager** to not pause location updates, location updates can be paused automatically by iOS or Mac Catalyst when the location data is unlikely to change. In this app, we don't want that to happen because we want to save the location multiple times so that we can establish whether a user visits a particular location frequently.

If you recall from earlier, we already defined the service as a partial class with a partial method; now, we will finish the service by implementing the platform-specific pieces of the service. Let's set this up:

1. Create a new folder named **Services** in the **Platforms/iOS** folder.
2. Create a new class named **LocationTrackingService** in the **Services** folder.
3. Modify the class to match the following:

```
namespace MeTracker.Services;
public partial class LocationTrackingService : ILocationTrackingService
{
    partial void StartTrackingInternal()
    {
    }
}
```

4. Add a private field for **CLLocationManager**.
5. Create an instance of **CLLocationManager** in the **StartTrackingInternal** method.
6. Set **PausesLocationUpdatesAutomatically** to **false**.

Before we can start tracking the location of the user, we need to set the accuracy of the data that we want to receive from **CLLocationManager**.

We will also add an event handler to handle location updates.

7. Set **DesiredAccuracy** to **CLLocation.AccuracyBestForNavigation**. One of the constraints when running the app in the background is that **DesiredAccuracy** needs to be set to either **AccuracyBest** or **AccuracyBestForNavigation**.
8. Set **AllowBackgroundLocationUpdates** to **true** (as shown in the following code snippet) so that the location updates will continue, even when the app is running in the background.

Your changes should look like this:

```
CLLocation locationManager;
partial void StartTrackingInternal()
{
    locationManager = new CLLocationManager
    {
        PausesLocationUpdatesAutomatically = false,
        DesiredAccuracy = CLLocation.AccuracyBestForNavigation,
        AllowsBackgroundLocationUpdates = true
    };
    // Add code here
}
```

The next step is to ask the user for permission to track their location. We will request permission to track their location all the time, but the user has the option of only giving us permission to track their location when they are using the app. Because the user also has the option of denying us permission to track their location, we need to check this before we start. Let's set this up:

1. Add an event handler for **LocationsUpdated** just after the **// Add code here** comment. It should look like the code highlighted in the following snippet:

```
partial void StartTrackingInternal()
{
    locationManager = new CLLocationManager
    {
        PausesLocationUpdatesAutomatically = false,
        DesiredAccuracy = CLLocation.AccuracyBestForNavigation,
        AllowsBackgroundLocationUpdates = true
    };
    // Add code here
    locationManager.LocationsUpdated +=
    async (object sender, CLLocationLocationsUpdatedEventArgs e) =>
    {
        // Final block of code goes here
    };
};
```

2. After the event handler, call the **RequestAlwaysAuthorization** method of the instance that we recently created in **CLLocationManager**:

```
partial void StartTrackingInternal()
```

```
{
    locationManager = new CLLocationManager
    {
        PausesLocationUpdatesAutomatically = false,
        DesiredAccuracy = CLLocation.AccurracyBestForNavigation,
        AllowsBackgroundLocationUpdates = true
    };
    // Add code here
    locationManager.LocationsUpdated +=
async (object sender, CLLocationLocationsUpdatedEventArgs e) =>
    {
        // Final block of code goes here
    };
    locationManager.RequestAlwaysAuthorization();
};
```

3. Then, call the **StartUpdatingLocation** method of **locationManager**:

```
partial void StartTrackingInternal()
{
    locationManager = new CLLocationManager
    {
        PausesLocationUpdatesAutomatically = false,
        DesiredAccuracy = CLLocation.AccurracyBestForNavigation,
        AllowsBackgroundLocationUpdates = true
    };
    // Add code here
    locationManager.LocationsUpdated +=
async (object sender, CLLocationLocationsUpdatedEventArgs e) =>
    {
        // Final block of code goes here
    };
    locationManager.RequestAlwaysAuthorization();
    locationManager.StartUpdatingLocation();
};
```

TIP

*The higher the accuracy is, the higher the battery consumption. If we only want to track where the user has been and not how popular a place is, we could also set **AllowDeferredLocationUpdatesUntil**. This way, we can specify that the user has to move a specific distance before the location is updated. We can also specify how often we want the location to be updated using the **timeout** argument. The most power-efficient solution to track how long a user has been at a place is to use the **StartMonitoringVisits** method of **CLLocationManager**.*

Now, it's time to handle the **LocationsUpdated** event. Let's go through the following steps:

1. Add a private field called **locationRepository** that is of the **ILocationRepository** type.
2. Add a constructor that has **ILocationRepository** as a parameter. Set the value of the parameter to the **locationRepository** field. Your class should resemble the following code snippet:

```
using CoreLocation;
```

```

using MeTracker.Repositories;
namespace MeTracker.Services;
public partial class LocationTrackingService : ILocationTrackingService
{
    CLLocationManager locationManager;
    ILocationRepository locationRepository;
    public LocationTrackingService(ILocationRepository locationRepository)
    {
        this.locationRepository = locationRepository;
    }
    partial void StartTrackingInternal()
    {
        // Remainder of code omitted for brevity
    }
}

```

3. Read the latest location of the **Locations** property of **CLLocationsUpdatedEventArgs**.
4. Create an instance of **MeTracker.Models.Location** and pass the latitude and longitude of the latest location to it.
5. Save the location using the **SaveAsync** method of **ILocationRepository**.
6. The code should be placed after the **// Final block of code goes here** comment. It should look like the code shown in bold in the following fragment:

```

locationManager.LocationsUpdated +=
async (object sender, CLLocationsUpdatedEventArgs e) =>
{
    // Final block of code goes here
    var lastLocation = e.Locations.Last();
    var newLocation = new Models.Location(lastLocation.Coordinate.Latitude, lastLocation.Coordi
        await locationRepository.SaveAsync(newLocation);
};

```

With that, we have completed the tracking part of the app for iOS. The implementation is identical for Mac Catalyst; you can either repeat the steps in this section for Mac Catalyst (but create the file as **Platforms/MacCatalyst/Services** instead of **Platforms/iOS/Services**) or copy the **Platforms/iOS/Services/LocationTrackingService.cs** file to the **Platforms/MacCatalyst/Services** folder.

Now, we will implement background tracking for Android, following which we will visualize the location tracking data.

Background location tracking with Android

The Android way to carry out background updates is very different from how we implemented it with iOS. With Android, we need to create a **JobService** class and schedule it.

Creating a background job

To track the location of users in the background, we need to create a background job. A background job is used by the OS to allow developers to execute code even when the app is not in the foreground or visible on

the screen. Follow these steps to create a background job to capture a user's location:

1. Create a new folder named **Services** in the **Platforms/Android** folder.
2. Create a new class called **LocationJobService** in the **Services** folder.
3. Make the class inherit from **Android.App.Job.JobService** as a base class.
4. Add **using Android.App.Job** and **using Android.App.Job** declarations to the top of the file.
5. Implement the **OnStartJob** and **OnStopJob** abstract methods, as shown in the following code snippet:

```
using Android.App;
using Android.App.Job;
namespace MeTracker.Platforms.Android.Services;
internal class LocationJobService : JobService
{
    public override bool OnStartJob(JobParameters @params)
    {
        return true;
    }
    public override bool OnStopJob(JobParameters @params)
    {
        return true;
    }
}
```

All the Android services in the app need to be added to the **AndroidManifest.xml** file. We don't have to do this manually; instead, we can add an attribute to the **LocationJobService** class, which will then be generated in the **AndroidManifest.xml** file. We will use the **Name** and **Permission** properties to set the required information, as shown in the following code snippet:

```
[Service(Name = "MeTracker.Platforms.Android.Services.LocationJobService", Permission = "android.p
internal class LocationJobService : JobService
```

Scheduling a background job

When we have created a job, we need to schedule it. We will do this from **LocationTrackingService** in the **Platforms/Android** folder. To configure the job, we will use the **JobInfo.Builder** class.

We will use the **SetPersisted** method to ensure that the job starts again after a reboot. This is why we added the **RECEIVE_BOOT_COMPLETED** permission earlier.

To schedule a job, at least one constraint is needed. In this case, we will use **SetOverrideDeadline**. This will specify that the job needs to run before the specified time (in milliseconds) has elapsed.

The **SetRequiresDeviceIdle** method can be used to make sure that a job only runs when the device is not being used by a user. We could pass **true**

to the method if we want to make sure that we don't slow down the device when the user is using it.

The **SetRequiresBatteryNotLow** method can be used to specify that a job should not run when the battery level is low. We recommend that this always be set to **true** if you don't have a good reason to run the job when the battery is low. This is because we don't want our applications to drain the user's battery.

So, let's implement **LocationTrackingService**. Follow these steps to do so:

1. Create a new class named **LocationTrackingService** in the **Platforms/Android/Services** folder.
2. Modify the class to look like the following:

```
namespace MeTracker.Services;
public partial class LocationTrackingService : ILocationTrackingService
{
    partial void StartTrackingInternal()
    {
    }
}
```

3. Create a **JobInfo.Builder** class based on an ID that we'll specify (we will use **1** here) and the component name (which we'll create from the application context and the Java class) in the **StartTrackingInternal** method. The component name is used to specify which code will run during the job.
4. Use the **SetOverrideDeadline** method and pass **1000** to it to make the job run before 1 second has elapsed from when the job was created.
5. Use the **SetPersisted** method and pass **true** to make the job persist even after the device is rebooted.
6. Use the **SetRequiresDeviceIdle** method and pass **false** so that the job will run even when a user is using the device.
7. Use the **SetRequiresBatteryLow** method and pass **true** to make sure that we don't drain the user's battery. This method was added in Android API level 26.

The code for **LocationTrackingService** should now look like this:

```
using Android.App.Job;
using Android.Content;
using MeTracker.Platforms.Android.Services;
namespace MeTracker.Services;
public partial class LocationTrackingService : ILocationTrackingService
{
    partial void StartTrackingInternal()
    {
        var javaClass = Java.Lang.Class.FromType(typeof(LocationJobService));
        var componentName = new ComponentName(global::Android.App.Application.Context, javaClass);
        var jobBuilder = new JobInfo.Builder(1, componentName);
        jobBuilder.SetOverrideDeadline(1000);
        jobBuilder.SetPersisted(true);
        jobBuilder.SetRequiresDeviceIdle(false);
        jobBuilder.SetRequiresBatteryNotLow(true);
        var jobInfo = jobBuilder.Build();
    }
}
```

```
    }
}
```

The last step in the **StartTrackingInternal** method is to schedule the job with the system using **JobScheduler**. The **JobScheduler** service is an Android system service. To get an instance of a system service, we will use the application context. Follow these steps to do so:

1. Use the **GetSystemService** method on **Application.Context** to get the **JobScheduler** service.
2. Cast the result to **JobScheduler**.
3. Use the **Schedule** method on the **JobScheduler** class and pass the **JobInfo** object to schedule the job, as shown in the following code snippet:

```
var jobScheduler = (JobScheduler)global::Android.App.Application.Context.GetService(Context.JobScheduler.SCHEDULE);
jobScheduler.Schedule(jobInfo);
```

Now that the job is scheduled, we can start receiving location updates; let's work on that next.

Subscribing to location updates

Once we have scheduled the job, we can write the code to specify what the job should do – that is, track the location of a user. To do this, we will use **LocationManager**, which is a **SystemService** class. With **LocationManager**, we can either request a single location update or we can subscribe to location updates. In this case, we want to subscribe to location updates.

We will start by creating an instance of the **ILocationRepository** interface. We will use this to save the locations to the SQLite database. Let's set this up:

1. Create a constructor for **LocationJobService**.
2. Create a **private** read-only field for the **ILocationRepository** interface called **locationRepository**.
3. Use **Services.GetService<T>** in the constructor to create an instance of **ILocationRepository**, as shown in the following code snippet:

```
private ILocationRepository locationRepository;
public LocationJobService()
{
    locationRepository = MauiApplication.Current.Services.GetService<ILocationRepository>();
}
```

Before we subscribe to location updates, we will add a listener. To do this, we will use the **Android.Locations.ILocationListener** interface.

Follow these steps:

1. Add the **Android.Locations.ILocationListener** interface to **LocationJobService**.
2. Add the following namespace declarations to the top of the file:

```
using Android.Content;
using Android.Locations;
using Android.OS;
using Android.Runtime;
using MeTracker.Repositories;
```

3. Implement the interface and remove all instances of **throw new NotImplementedException()**. This is added to the methods when you let Visual Studio generate an implementation of the interface. The method implementations should look like those in the following code snippet:

```
public override bool OnStartJob(JobParameters @params)
{
    return true;
}
public void OnLocationChanged(global::Android.Locations.Location location) { }
public override bool OnStopJob(JobParameters @params) => true;
public void OnStatusChanged(string provider, [GeneratedEnum] Availability status, Bundle ex
public void OnProviderDisabled(string provider) { }
public void OnProviderEnabled(string provider) { }
```

4. In the **OnLocationChanged** method, map the **Android.Locations.Location** object to the **Model.Location** object.
5. Use the **SaveAsync** method on the **LocationRepository** class, as shown in the following code snippet:

```
public void OnLocationChanged(Android.Locations.Location location)
{
    var newLocation = new Models.Location(location.Latitude, location.Longitude);
    locationRepository.SaveAsync(newLocation);
}
```

Now that we have created a listener, we can subscribe to location updates. Follow these steps to do so:

1. Create a **static** field of the **LocationManager** type named **locationManager**. Make sure it has the same lifetime as the app.
2. It is possible in Android that **JobService** will start before **MainView** is displayed and we request location permissions. To prevent any errors due to missing permissions, we will check for them first:

```
public override bool OnStartJob(JobParameters @params)
{
    PermissionStatus status = PermissionStatus.Unknown;
    Task.Run(async () => status = await AppPermissions.CheckRequiredPermissionAsync()).Wait();
    if (status == PermissionStatus.Granted)
    {
    }
}
```

We run **CheckRequiredPermissionsAsync** inside a **Task.Run** instance because it's an **async** call, and we can't add **async** to the method because the return type is incompatible. The call to **Wait** turns the **async** call into a synchronous one. If the result is **Granted**, then we can continue.

3. Go to the **StartJob** method in **LocationJobService**. Get **LocationManager** by using **GetSystemService** on **ApplicationContext**.
4. To subscribe for location updates, use the **RequestLocationUpdates** method, as shown in the following code snippet:

```
public override bool OnStartJob(JobParameters @params)
{
    PermissionStatus status = PermissionStatus.Unknown;
    Task.Run(async () => status = await AppPermissions.CheckRequiredPermissionAsync()).Wait();
    if (status == PermissionStatus.Granted)
    {
        locationManager = (LocationManager)ApplicationContext.GetService (Context.Locati
        locationManager.RequestLocationUpdates (LocationManager.GpsProvider, 1000L, 0.1f, this)
        return true;
    }
    return false;
}
```

The first argument that we pass to the **RequestLocationUpdates** method ensures that we get locations from the GPS. The second ensures that at least **1000** milliseconds will elapse between location updates. The third argument ensures that the user moves at least **0.1** meters to get a location update. The last argument specifies which listener we should use.

Because the current class implements the **Android.Locations.ILocationListener** interface, we will pass **this**.

Now that we have collected location data on from the user and stored it in our SQLite database, we can now display that data on a map.

Creating a heat map

To visualize the data that we have collected, we will create a heat map. We will add lots of dots to the map and make them different colors, based on how much time a user spends in a particular place. The most popular places will have a warm color, while the least popular places will have a cold color.

Before we add the dots to the map, we need to get all locations from the repository.

Adding the **GetAllAsync** method to **LocationRepository**

In order to visualize the data, we need to write some code so that location data can be read from the database. Let's set this up:

1. Open the **ILocationRepository.cs** file.
2. Add a **GetAllAsync** method that returns a list of **Location** objects, using the following code:

```
Task<List<Models.Location>> GetAllAsync();
```

3. Open the **LocationRepository.cs** file, which implements **ILocationRepository**.
4. Implement the new **GetAllAsync** method and return all the saved locations in the database, as shown in the following code snippet:

```
public async Task<List<Location>> GetAllAsync()
{
    await CreateConnectionAsync();
    var locations = await connection.Table<Location> ().ToListAsync();
    return locations;
}
```

Preparing the data for visualization

Before we can visualize the data on the map, we need to prepare the data. The first thing we will do is create a new model that we can use for the prepared data. Let's set this up:

1. In the **Models** folder, create a new class called **Point**.
2. Add properties for **Location**, **Count**, and **Heat**, as shown in the following code snippet:

```
namespace MeTracker.Models{
public class Point
{
    public Location Location { get; set; }
    public int Count { get; set; } = 1;
    public Color Heat { get; set; }
}
}
```

MainViewModel will store the locations that we will find later on. Let's add a property for storing points.

3. Open the **MainViewModel** class.
4. Add a **private** field called **points**, which is of the **List<Point>** type.
5. Add the **ObservableProperty** attribute to the **points** field, as shown in the following code snippet:

```
[ObservableProperty]
private List<Models.Point> points;
```

Now that we have storage for our points, we must add some code so that we can add locations. We will do this by implementing the **LoadDataAsync** method of the **MainViewModel** class and making sure that it is called on the main thread, right after location tracking has started.

The first thing we will do is group the saved locations so that all locations within 200 meters will be handled as one point. We will track how many times we have logged a position within that point so that we can decide which color the point will be on the map. Let's set this up:

1. Add an **async** method called **LoadDataAsync**. This returns a **Task** object to **Main ViewModel**.
2. Call the **LoadDataAsync** method from the constructor after the call to the **StartTracking** method on **ILocationTrackingService**, as shown in the following code snippet:

```
public MainViewModel(ILocationTrackingService locationTrackingService, ILocationRepository loca
{
    this.locationTrackingService = locationTrackingService;
    this.locationRepository = locationRepository;
    MainThread.BeginInvokeOnMainThread(async() =>
    {
        locationTrackingService.StartTracking();
        await LoadDataAsync();
    });
}
```

The first step of the **LoadDataAsync** method is to read all tracked locations from the SQLite database. When we have all the locations, we will loop through them and create points.

To calculate the distance between a location and a point, we will use the **CalculateDistance** method, as shown in the following code snippet:

```
private async Task LoadDataAsync()
{
    var locations = await locationRepository.GetAll();
    var pointList = new List<Models.Point>();
    foreach (var location in locations)
    {
        //If no points exist, create a new one and continue to the next location in the list
        if (!pointList.Any())
        {
            pointList.Add(new Models.Point() { Location = location });
            continue;
        }
        var pointFound = false;
        //try to find a point for the current location
        foreach (var point in pointList)
        {
            var distance = Location.CalculateDistance(
                new Location(point.Location.Latitude, point.Location.Longitude),
                new Location(location.Latitude, location.Longitude),
                DistanceUnits.Kilometers);
            if (distance < 0.2)
            {
                pointFound = true;
                point.Count++;
                break;
            }
        }
        //if no point is found, add a new Point to the list of points
        if (!pointFound)
        {
            pointList.Add(new Models.Point() { Location = location });
        }
    }
}
```

```
// Next section of code goes here
}
}
```

When we have a list of points, we can calculate the heat color for each point. We are going to use the **hue**, **saturation**, and **lightness (HSL)** representation of a color, as described here:

- **Hue:** Hue is a degree on the color wheel that goes from 0 to 360, with 0 being red and 240 being blue. Because we want our most popular places to be red (hot) and our least popular places to be blue (cold), we will calculate a value between 0 and 240 for each point, based on how many times the user has been to that point. This means that we will only use two-thirds of the scale.
- **Saturation:** Saturation is a percentage value: 0% is a shade of gray, while 100% is full color. In our app, we will always use 100% (this will be represented as **1** in the code).
- **Lightness:** Lightness is a percentage value of the amount of light: 0% is black and 100% is white. We want it to be neutral, so we will use 50% (this will be represented as **0.5** in the code).

The first thing that we need to do is find out how many times the user has been to the most popular and least popular places. Let's take a look:

1. First, check that the list of points is not empty.
2. Get the **Min** and **Max** values for the **Count** property in the list of points.
3. Calculate the difference between the minimum and the maximum values.
4. The code should be added after the **// Next section of code goes here** comment at the bottom of the **LoadDataAsync** method, as shown in the following code snippet:

```
private async Task LoadDataAsync()
{
    // The rest of the method has been omitted for brevity
    // Next section of code goes here
    if (pointList == null || !pointList.Any())
    {
        return;
    }
    var pointMax = pointList.Select(x => x.Count).Max();
    var pointMin = pointList.Select(x => x.Count).Min();
    var diff = (float)(pointMax - pointMin);
    // Last section of code goes here
}
```

Now, we can calculate the heat for each point, as follows:

1. Loop through all the points.
2. The following code should be added after the **// Last section of code goes here** comment, at the bottom of the **LoadDataAsync()** method (this is highlighted in the following code snippet):

```
private async Task LoadDataAsync()
{
```



```
// The rest of the method has been omitted for brevity
// Next section of code goes here
if (pointList == null || !pointList.Any())
{
    return;
}
var pointMax = pointList.Select(x => x.Count).Max();
var pointMin = pointList.Select(x => x.Count).Min();
var diff = (float)(pointMax - pointMin);
// Last section of code goes here
foreach (var point in pointList)
{
    var heat = (2f / 3f) - ((float)point.Count / diff);
    point.Heat = Color.FromHsla(heat, 1, 0.5);
}
Points = pointList;
}
```

That's all we need to do to set up location tracking in the **MeTracker** project. Now, let's turn our attention to visualizing the data we receive.

Adding data visualizations

In .NET MAUI, the **Map** control can render additional information over the map. This includes pins and custom shapes, which are called **MapElements**. We could simply add each location that is stored in the repository as a pin; however, to get the heat map, we want to add a colored dot to the map for each location, so we will use **MapElements** for each location.

If the **MapElements** property were **BindableProperty**, we could use a converter to map the **MainViewModel Points** property to the map's **MapElements** property in a binding. But **MapElements** is not a bindable property, so it won't be that easy.

Let's start by creating a custom map control.

Creating a custom control for the map

To show the heat map on our map, we will create a new control. Since **Map** is a sealed class, we won't be able to subclass it directly; instead, we will encapsulate the **Map** control inside **ContentView** with **BindableProperty** to get access to the **Points** data from **ViewModel**.

Follow these steps to create the custom control:

1. Create a new folder called **Controls**.
2. Create a new class called **CustomMap**.
3. Add **ContentView** as a base class to the new class, as shown in the following code snippet:

```
namespace MeTracker.Controls;
public class CustomMap : ContentView
{
    public CustomMap()
```

```

    {
    }
}

```

Now, we need to add the **Map** control to the custom control. Follow these steps to add the **Map** control:

1. Derive the **CustomMap** control from the .NET MAUI **Map** control, as highlighted in the following code snippet:

```

using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
namespace MeTracker.Controls;
public class CustomMap : Map
{
    public CustomMap()
    {
    }
}

```

2. Initialize the map in the constructor, as shown with the new changes highlighted:

```

public CustomMap()
{
    IsScrollEnabled = true;
    IsShowingUser = true;
}

```

If we want to have properties that we want to bind data to, we need to create a **BindableProperty** class. This should be a public static field in the class. We also need to create a regular property to hold the value. The naming of the properties is really important. The name of **BindableProperty** needs to be **{NameOfTheProperty}Property**; for example, the name of **BindableProperty** that we will create in the following steps will be **PointsProperty** because the name of the property is **Points**. A **BindableProperty** is created using the static **Create** method on the **BindableProperty** class. This requires at least four arguments, as follows:

- **propertyName**: This is the name of the property as a string.
- **returnType**: This is the type that will be returned from the property.
- **declaringType**: This is the type of the class that **BindableProperty** is declared in.
- **defaultValue**: This is the default value that will be returned if no value is set. This is an optional argument. If it is not set, .NET MAUI will use **null** as a default value.

The **set** and **get** methods for the property will call methods in the base class to set or get values from **BindableProperty**:

1. Create **BindableProperty** called **PointsProperty**, as shown in the following code snippet:

```
public static BindableProperty PointsProperty = BindableProperty.Create(nameof(Points), typeof(
```

2. Create a property of the `List<Models.Point>` type called **Points**.

Remember to cast the result of `GetValue` so that it's the same type as the property. We need to do this because `GetValue` will return the value as a **type** object:

```
public List<Models.Point> Points
{
    get => GetValue(PointsProperty) as List<Models.Point>;
    set => SetValue(PointsProperty, value);
}
```

In order to display **Points**, we need to convert them to **MapElements**. This is accomplished using a **BindingProperty** event called **PropertyChanged**. **PropertyChanged** is fired every time **BindingProperty** changes. To add the event and convert **Points** to **MapElements**, add the following high-lighted code to the class:

```
public readonly static BindableProperty PointsProperty = BindableProperty.Create(nameof(Points), t
private static void OnPointsChanged(BindableObject bindable, object oldValue, object newValue)
{
    var map = bindable as Map;
    if (newValue == null) return;
    if (map == null) return;
    foreach (var point in newValue as List<Models.Point>)
    {
        // Instantiate a Circle
        Circle circle = new()
        {
            Center = new Location(point.Location.Latitude, point.Location.Longitude),
            Radius = new Distance(200),
            StrokeColor = Color.FromArgb("#88FF0000"),
            StrokeWidth = 0,
            FillColor = point.Heat
        };
        // Add the Circle to the map's MapElements collection
        map.MapElements.Add(circle);
    }
}
public List<Models.Point> Points
{
    get => GetValue(PointsProperty) as List<Models.Point>;
    set => SetValue(PointsProperty, value);
}
```

Now that we've created a custom map control, we will use it to replace the **Map** control in **MainView**. Follow these steps:

1. In the **MainView.xaml** file, declare the namespace for the custom control.
2. Replace the **Map** control with the new control that we have created.
3. Add a binding to the **Points** property in **MainViewModel**, as shown in the following code snippet:

```
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms" xmlns:x="http://schemas.microsoft.com
xmlns:map="clr-namespace:MeTracker.Controls;"
x:Class="MeTracker.Views.MainView">
<map:CustomMap x:Name="Map" Points="{Binding Points}" />
</ContentPage>
```

This concludes this section on how to extend the **Maps** control. The final step for our app is to refresh the map when the app resumes.

Refreshing the map when the app resumes

The last thing we will do is make sure that the map is up to date with the latest points when the app is resumed. The easiest way to do this is to set the **MainPage** property in the **App.xaml.cs** file to a new instance of **AppShell1**, in the same way as the constructor, as shown in the following code snippet:

```
protected override void OnResume()
{
    base.OnResume();
    MainPage = new AppShell1();
}
```

The **MeTracker** app is now complete – try it out. A sample screenshot is shown in *Figure 7.18*:

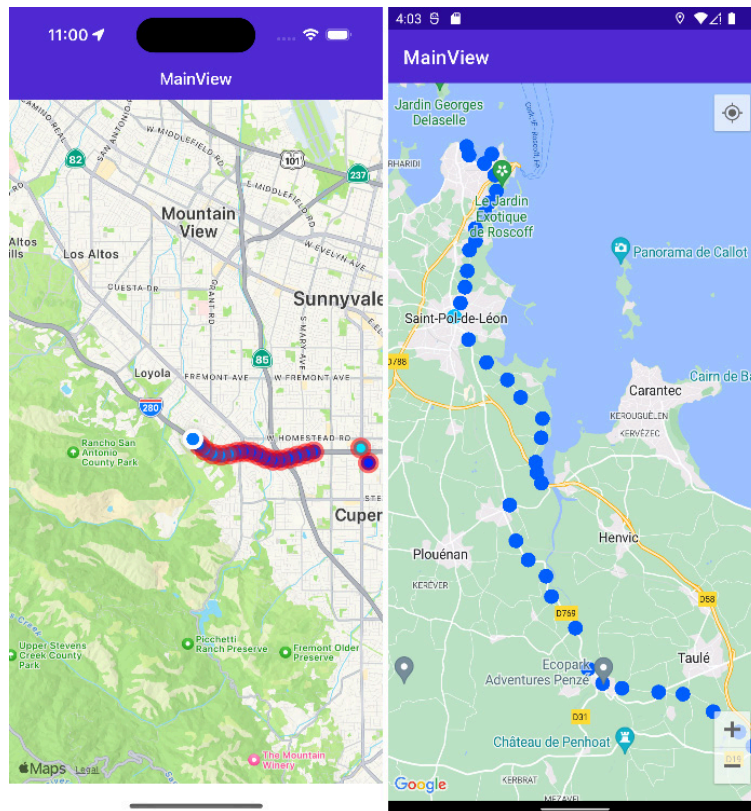


Figure 7.18 – MeTracker on iOS and Android

Summary

In this chapter, we built an app for iOS, Mac Catalyst, and Android that tracked the location of a user. When we built the app, we learned how to use maps in .NET MAUI and how to use location tracking when it's running in the background. We also learned how to extend .NET MAUI with custom controls. With this knowledge, we can create applications that perform other tasks in the background. We also learned how to extend most controls in .NET MAUI.

Here are some ways you could extend this app even further:

- Right now, the app updates the map location when the app resumes. How could you update the map when the location changes?
- Add a view that lists all locations from the database. Allow the user to remove a location from the list.

The next project will be a weather app. In the next chapter, we will use an existing weather service API to retrieve weather data and then display that data in the app.